Week 10 part 2

✅ Message Passing

- Synchronization
- Pipe
- Fork and a pipe
- why we need dup?
- read write from to pipe

Content

✅ Message Passing

- OS can do synchronization for us

Message Passing

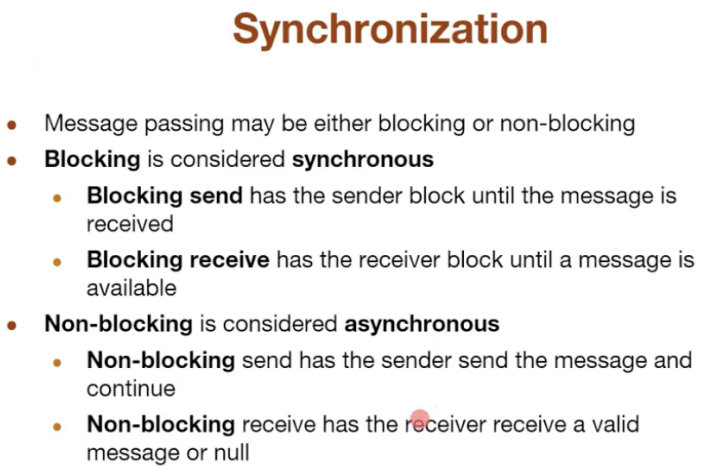
- Message-based interprocess communication
- IPC facility provides two operations:
  - **send**(*message*) – message size fixed or variable
  - **receive**(*message*)
- If *P* and *Q* wish to communicate, they need to:
  - establish a *communication link* between them
  - exchange messages via send/receive
- Messages can be passed in one direction at a time
  - One process is the sender and the other is the receiver
- Message passing can be bidirectional
  - Each process can act as either a sender or a receiver
- Popular implementation is a pipe
  - A region of memory protected by the OS that serves as a buffer, allowing two or more processes to exchange data

Overview idea of message passing

- it has to provide at least 2 functions for you

    - send → to send messge (might be write)
    - receive → to receive message (might be read)
    - but the function name can be any
- actually, we can use message passing
    - one way
    - bidirectional
- popular implemetaion is a pipe
    - pipe is a region of memory protected by os.
    - for msg passing → OS play the role and it can do the synchronization for us

📍 Synchronization



- by default OS provides the syncronous or blocking command for us
    - the command can be blocking send or blocking receive
    - **blocking send** mean when the sender send the data → it wait until the receiver read the data and it can continue
    - **blocking receiver** mean if the receiver go to read the data and the new data still not coming, the receiver then wait until the new data is send.

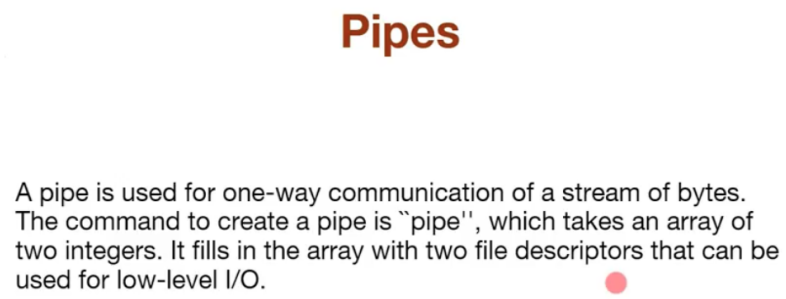
  using this → the receiver never lost a data or get book-ahead data

  if programmer don't want to use blocking → you can choose non-blocking instead

- **non blocking send** - after sending a data, the process continue without having to wait
- **non blocking receive** - if the data is not comming, receiver can go

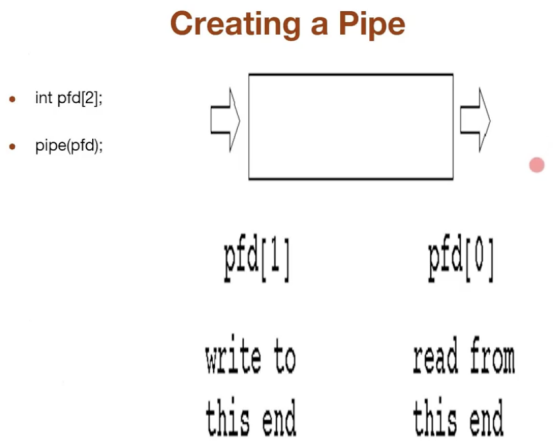
our class focus on blocking one

📍 Pipe



- pipe is used for one-way communication of a stream of bytes.
- The command to create pipe is “pipe”, which takes an array of 2 integers. It fills in the array with 2 file descriptors that can be used for low-level I/O.
  - we use pipe system call to create pipe
  - parameter → array of 2 integer
  - return 2 file descriptor
  - using pipe is like using low level file i/o
  - what is file descriptor?
    - int that represent the file
      - when you read or write, you can use this int instead of file name
    - when you write program → you only write file name the first time
    - use filename only the first time → use file descriptor

Creating a Pipe



- file descriptor that return at index 0 → read from file (file descriptor for reading)
- file descriptor that return at index 1 → write to file
- you get 2, one for read, one for write.

I/O with a pipe

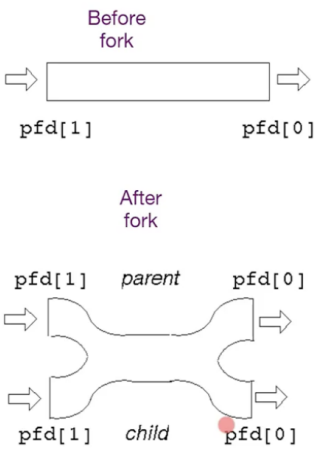
- These 2 file descriptors can be used for block I/O
  - write(pfd[1], buf, size)
    - low level i/o to write data into a file
    - here, yu write to write end of a pipe
    - the data is store in array name buf
    - you specify the size of array of data you want to write
    - size here can be vary → you can specify
  - read(pfd[0], buf, SIZE)
    - SIZE → you can read not more than this size
    - let said buf array is 1000 → you can't read more than 1000 bytes
    - what if the data is larger than 1000 → use while loop to read

📍 Fork and pipe

- if parent and child want to communicate through a pipe, should it fork first or create pipe first.

- A single process would not use a pipe.
- They are used when two processes wish to communicate in a one-way fashion.
- A process splits in two using ``fork''.
- A pipe opened before the fork becomes shared between the two processes.

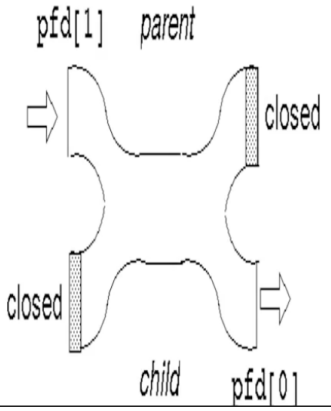
- If parent fork before call pipe → the pipe that parent create will never copy to the child.
    - **so create pipe first, so the child can access pipe too.**



This gives two read ends and two write ends. Either process can write into the pipe, and either can read from it. Which process will get what is not known.

- Pipe before fork and after fork
- And Pipe is one way communication, you have to specify which one is sender and which one is receiver
    - ex, parent is sender and child is receiver → the read end of parent would not be needed,(close read end of parent), close write end of child too
        - get pipe from parent to child
        - parent write, child read
    - Get this

Fork and a pipe (Contd.)



- For predictable behaviour, one of the processes must close its read end, and the other must close its write end. Then it will become a simple pipeline again.
- Suppose the parent wants to write down a pipeline to a child. The parent closes its read end, and writes into the other end. The child closes its write end and reads from the other end.

- if not close, sometime, it not work correctly
- for this pipe, it write on same machine
  - for socket → different machine on network
  - you can think of network as virtual file
  - socket mean 2 thing → ip address and port number, it also have to pass through network layer
  - socket library of java, marchel link?
  - for socket → use network protocol

Example

pipe() Example (1)

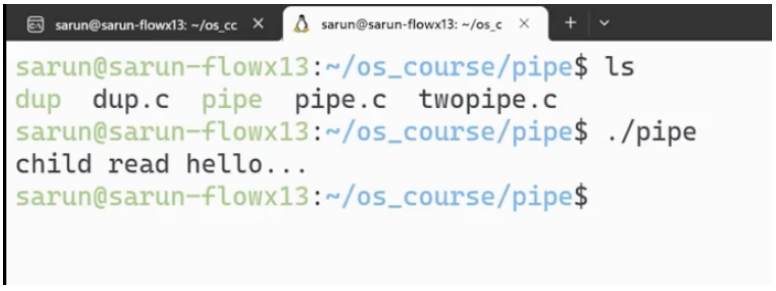
```
#include <stdio.h>
#define SIZE 1024
int main(int argc, char **argv)
{
  int pfd[2];
  int nread;
  int pid;
  char buf[SIZE];
  if (pipe(pfd) == -1)
  {
    perror("pipe failed");
    exit(1);
  }
  if ((pid = fork()) < 0)
  {
    perror("fork failed");
    exit(2);
  }
```

- if you can’t pipe, return -1

```
if (pid == 0)
  {
    /* child */
    close(pfd[1]);
    while ((nread =read(pfd[0], buf, SIZE)) != 0)
      printf("child read %s\n", buf);
    close(pfd[0]);
  } else {
    /* parent */
          close(pfd[0]);
          strcpy(buf, "hello...");
    /* include null terminator in write */
          write(pfd[1], buf,
          strlen(buf)+1);
          close(pfd[1]);
          wait();
  }
  exit(0);
```

- read until nread is 0 (no more data)
- after read all, close read end
- strlen(buf)+1 because strlen return without \n

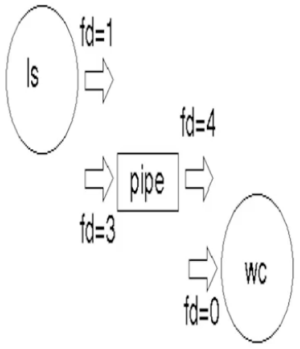
Output



📍Why we need dup?

Example: why we need dup?

- To implement ``ls | wc'' the shell will have created a pipe and then forked.
- The parent will exec to be replaced by ``ls'', and the child will exec to be replaced by ``wc''
- The write end of the pipe may be descriptor 3 and the read end may be descriptor 4.
- ``ls'' normally writes to 1 and ``wc'' normally reads from 0.
- How do these get matched up?



point is we need to link 1 with 3, and link 4 to 0

wc → word count

ls | wc → output of ls be an input to wc

- using pipe we direct the output of ls from monitor to ls
- we redirect output of wc to a pipe.
- pipe is the way to redirect input and output

```
sarun@sarun-flowx13:~/os_course/pipe$ ls > test.txt
```

- way to store output in file (overwritten)

>> → append file

---

what if we want to write a prograc ls | wc

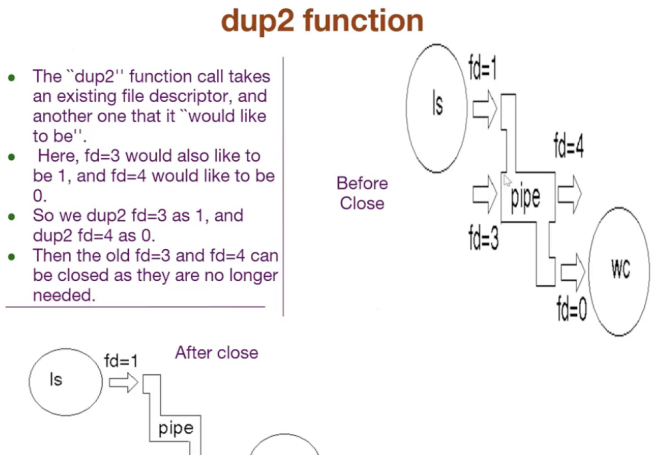
- like the parent exec ls and send output to child to exec wc

by default → the file descriptor 0 is stardard input, file descriptor 1 is starndard output, file descriptor 2 is standard error

- this are system file descriptor
- when you create a file or open, you wil never get file descriptor 0(keyboard), 1(monitor) or 2(monitor-youcan redirect to printer) because it reserved by OS.

point is we need to link 1 with 3, and link 4 to 0

- use dup2 function



- we need to link 1 to 3 and 4 to 0. so the data will be pass from ls to wc
- how to do that?
- use dup2 function

dup2() Example(2)

```
if (pid == 0)
 {
   close(pfd[0]);
   dup2(pfd[1], 1 );
   close(pfd[1]);
   execlp("ls", "ls",  (char *) 0);
   perror("ls failed");
   exit(3);
 } else {
   close(pfd[1]);
   dup2(pfd[0], 0);
   close(pfd[0]);
   execlp("wc", "wc", (char *) 0);
   perror("wc failed");
   exit(4);
 }
 exit(0);
}
```

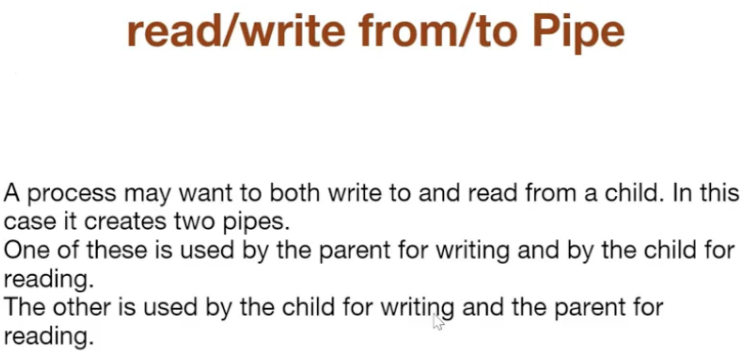
- dup write end of the pipe to file descriptor number 1
- dup read end to file descriptor number 0
- for this program child will exec ls and send the data to parent. (data from ls will be send through wc exec by parent)

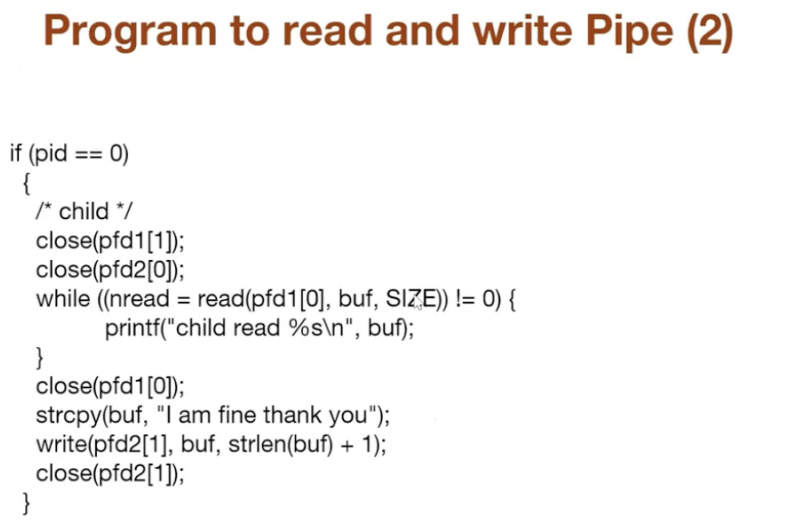
- when ls is executed, insted of write to file descriptor 1, it will write to write end of the pipe.
- wx read from read end of the pipe.

📍 Read and write from to pipe

pipe that we use is 1 way, if we want 2 way create 2 pipe.

- create 2 pipe to create bi relational





Lab7 (pipe)

1. Write a C program as follows. The program receives a number from command line when a user runs the program. The parent then forks a child to calculate the summation from 1 to the number that the user specified. The child sends the result back to the parent via the pipe. The parents then prints the result.

Note: One way to convert an integer to string is to use sprintf() function as follows

sprintf(str, "%d", sum);

This statement converts the integer store in the variable sum to string and store the result in the variable str.

The example output of the program is as follows:

./lab7_pipe 5
Parent: waiting for my child
Child: I am calculating
Child: the result is 15
Child: I am sending data
Child: Goodbye
Parent: sum from my child is 15
Parent: Goodbye

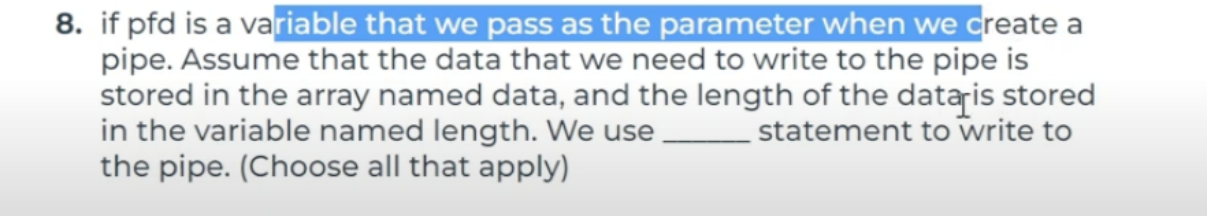
```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#define SIZE 1024
int main(int argc, char *argv[])
{
    int pfd[2];
    int nread;
    int pid;
    char buf[SIZE];
    int num;
    if (argc != 2) {
        printf("Usage lab7_pipe <number>\n");
        exit(0);
    }
    num = atoi(argv[1]);
    if (pipe(pfd) == -1)
    {
        perror("pipe failed");
        exit(1);
```

```
    }
    if ((pid = fork()) < 0)
    {
        perror("fork failed");
        exit(2);
    }

    if (pid == 0)
    {
        /* child */
        close(pfd[0]);
        printf("Child: I am calculating\n");
        int i, sum = 0;
        for(i = 1; i <= num; i++) {
            sum += i;
        }
        printf("Child: the result is %d\n", sum);
        sprintf(buf, "%d", sum);
        printf("Child: I am sending data\n");
        write(pfd[1], buf,strlen(buf)+1);
        printf("Child: Goodbye\n");
        close(pfd[1]);
  } else {
    /* parent */
        close(pfd[1]);
        printf("Parent: waiting for my child\n");
        wait(NULL);
        nread = read(pfd[0], buf, SIZE);
        int sum;
        sum = atoi(buf);
        printf("Parent: sum from my child is %d\n", sum);
        printf("Parent: Goodbye\n");
        close(pfd[0]);
    }
  exit(0);
}
```

Quiz

1. The message passign function that is used to send data must be named send()
    - False
2. Message passing can only be unidirectional
    - False (can be both)
3. What is (are) considered the synchronous passing
    - A and C
    - A → blocking send (a sender waits until the receiver gets the message.)
    - C → blocking receive (a receiver waits until a message is availiable)
4. The funciton to create a pipe is
    - pipe()
5. The parameter that will be passed to the function in the previous question is
    - An array of integer that has 2 elements
6. We use concet of writing and reading a file when working with pipe.
    - pipe
7. After creatieng a pipe we get file descriptors and use them to work with the pipe
    - True

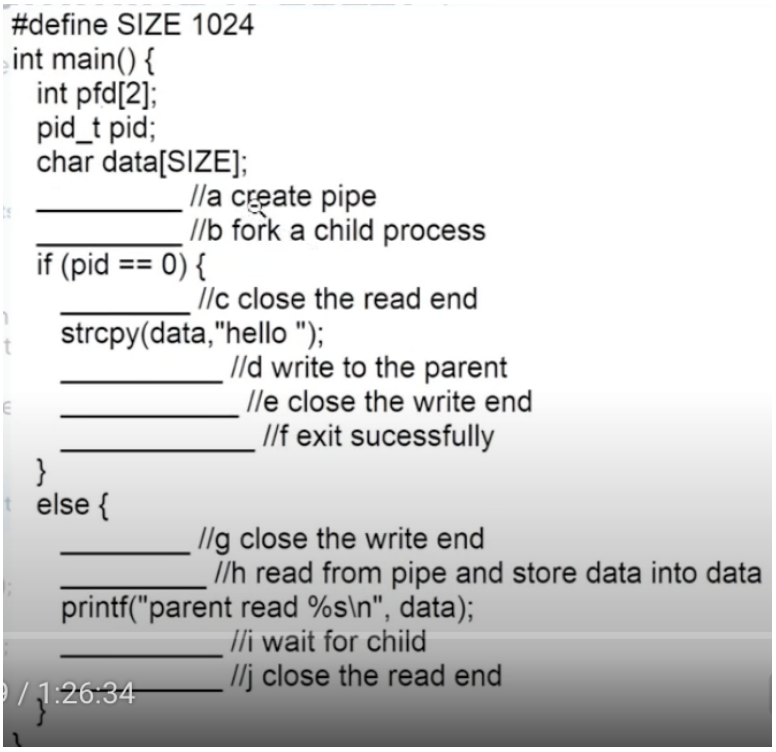


- write(pdf[1], data, length);

9. If a parent needs to communicate with its child, it will fork the child process and then create a pipe and use that pipe to communicate to the child.

- False

10. -19. Assuming that we need to write a program to send data from child process to parent



- //a → pipe(pfd)
- //b → pid=fork()
- //c → close(pdf[0])
- //d → write(pdf[1], data, strlen(buf)+1);
- //e → close(pfd[1]);
- //f → exit(0)
- //g → close(pdf[1])
- //h → read(pdf[0], data, SIZE);
- //i → wait(NULL)
- //j → close([pdf[0])

20. The file descriptor that represents standard input is
    - 0
21. The file descriptor that represents standard output is
    - 1
22. The default file descriotir that ls command uses to display data is
    - 1

23. File descriptors that we will get after creating a pipe must be greater than 3.
    - False
24. If we need to redirect the standard output to the write end of the pipe, the statement that we will use is
    - dup2(pfd[1], 1);
25. If we need to redirect the standard input to the read-end of the pipe, the staement htat we will use is
    - dup2([pfd[0], 0);